

PRACTICAL WEEK-5

TASK-1 : Implement a producer-consumer communication system using anonymous pipes where the parent process generates data and the child process consumes it. Measure the communication efficiency.

```
root@DESKTOP-QU0HVJI: ~/t × + ∨
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/time.h>

#define COUNT 100000

int main()
{
    int pipefd[2];
    pid_t pid;
    int data;
    long long sum = 0;
    struct timeval start, end;

    if (pipe(pipefd) == -1)
    {
        perror("pipe");
        return 1;
    }

    gettimeofday(&start, NULL);
    pid = fork();
    if (pid == -1)
    {
        perror("fork");
        return 1;
    }
    if (pid > 0)
    {
        /* Parent - Producer */
        close(pipefd[0]);
        for (int i = 0; i < COUNT; i++)
        {
            data = i;
            if (write(pipefd[1], &data, sizeof(data)) == -1)
            {
                perror("write");
                return 1;
            }
        }
        close(pipefd[1]);
        wait(NULL);
        gettimeofday(&end, NULL);

        double time_taken =
            (end.tv_sec - start.tv_sec) +
            (end.tv_usec - start.tv_usec) / 1000000.0;

        long total_bytes = COUNT * sizeof(int);
        double throughput = total_bytes / time_taken;

        printf("\nCommunication completed.\n");
        printf("Data transferred: %d integers\n", COUNT);
        printf("Time taken: %.6f seconds\n", time_taken);
        printf("Throughput: %.2f bytes/second\n", throughput);
    }
    else
    {
        /* Child - Consumer */
        close(pipefd[1]);
        for (int i = 0; i < COUNT; i++)
        {
            if (read(pipefd[0], &data, sizeof(data)) == -1)
            {
                perror("read");
                exit(1);
            }
            sum += data;
        }
        close(pipefd[0]);

        printf("Consumer received %d integers.\n", COUNT);
        printf("Sum of received data: %lld\n", sum);
    }
    return 0;
}
```

OUTPUT:

```
root@DESKTOP-QU0HVJI:~/practical/practical-5# nano producer_consumer.c
root@DESKTOP-QU0HVJI:~/practical/practical-5# gcc producer_consumer.c -o producer_consumer
root@DESKTOP-QU0HVJI:~/practical/practical-5# ./producer_consumer
Consumer received 100000 integers.
Sum of received data: 4999950000

Communication completed.
Data transferred: 100000 integers
Time taken: 0.045156 seconds
Throughput: 8858180.53 bytes/second
root@DESKTOP-QU0HVJI:~/practical/practical-5# |
```

REPORT: I implemented a producer-consumer communication system using an anonymous pipe, where the parent process generated data and the child process consumed it. The program successfully transferred 100,000 integers, and the communication time and throughput were measured. The program was successfully executed and tested in Ubuntu.

TASK-2:

Develop a program that executes the equivalent of the shell command: `ls -l | grep ".c"` using `fork()`, `pipe()`, `dup2()`, and `exec()` system calls.

```
root@DESKTOP-QU0HVJI: ~/l × + v
GNU nano 7.2 pipe_grep.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int pipefd[2];
    pid_t pid1, pid2;

    if (pipe(pipefd) == -1)
    {
        perror("pipe");
        return 1;
    }

    // First child: ls -l
    pid1 = fork();

    if (pid1 == -1)
    {
        perror("fork");
        return 1;
    }

    if (pid1 == 0)
    {
        close(pipefd[0]);
        dup2(pipefd[1], STDOUT_FILENO);
        close(pipefd[1]);
        execlp("ls", "ls", "-l", NULL);
        perror("execlp");
        exit(1);
    }

    // Second child: grep ".c"
    pid2 = fork();

    if (pid2 == -1)
    {
        perror("fork");
        return 1;
    }
}
```

OUTPUT:

```
root@DESKTOP-QU0HVJI:~/practical/practical-5# gcc pipe_grep.c -o pipe_grep
root@DESKTOP-QU0HVJI:~/practical/practical-5# ./pipe_grep
-rw-r--r-- 1 root root 1032 Sep 19 05:33 pipe_grep.c
-rwxr-xr-x 1 root root 16448 Sep 19 05:16 producer_consumer
-rw-r--r-- 1 root root 1780 Sep 19 05:16 producer_consumer.c
root@DESKTOP-QU0HVJI:~/practical/practical-5# ls -l | grep ".c"
ls: cannot access '-': No such file or directory
ls: cannot access 'l': No such file or directory
root@DESKTOP-QU0HVJI:~/practical/practical-5# |
```

REPORT: I developed a program to execute the equivalent of `ls -l | grep ".c"` using `fork()`, `pipe()`, `dup2()`, and `exec()` system calls. The output of `ls -l` was passed through the pipe to `grep` for filtering. The program was successfully compiled, executed, and verified in Ubuntu.

